

Yarn Architecture

In previous lectures, we have understood Hive parses and submit the queries to execution engines like MR, tez, or Hive on Spark. But these execution engines need resources to run the queries, so we need a platform that acts like a bank of hosts required to run our queries and these execution engines can approach that bank and ask for resources to execute their queries, now this bank can decide how many resources or what type of resources are required and for how much time. Let's take a deep dive to understand it better.

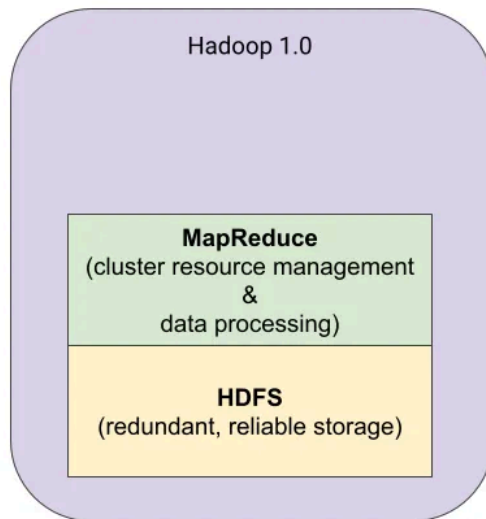
The primary goal of YARN is to decouple the functionalities of resource management and data processing into separate components. This allows a global resource manager that can support diverse data processing applications, such as MapReduce, Spark, Storm, Tez, and others, to run on a single cluster.

Before YARN was introduced, Apache Hadoop used a resource manager called MapReduce, which served as both a resource manager and a processing engine. This system was tightly coupled with the Hadoop Distributed File System (HDFS) and limited to running MapReduce jobs only. This made it difficult for users to run other types of applications on the Hadoop cluster.

To overcome these limitations, YARN was developed. YARN separates the processing engine and management functions of MapReduce, allowing it to support multiple processing engines and applications.

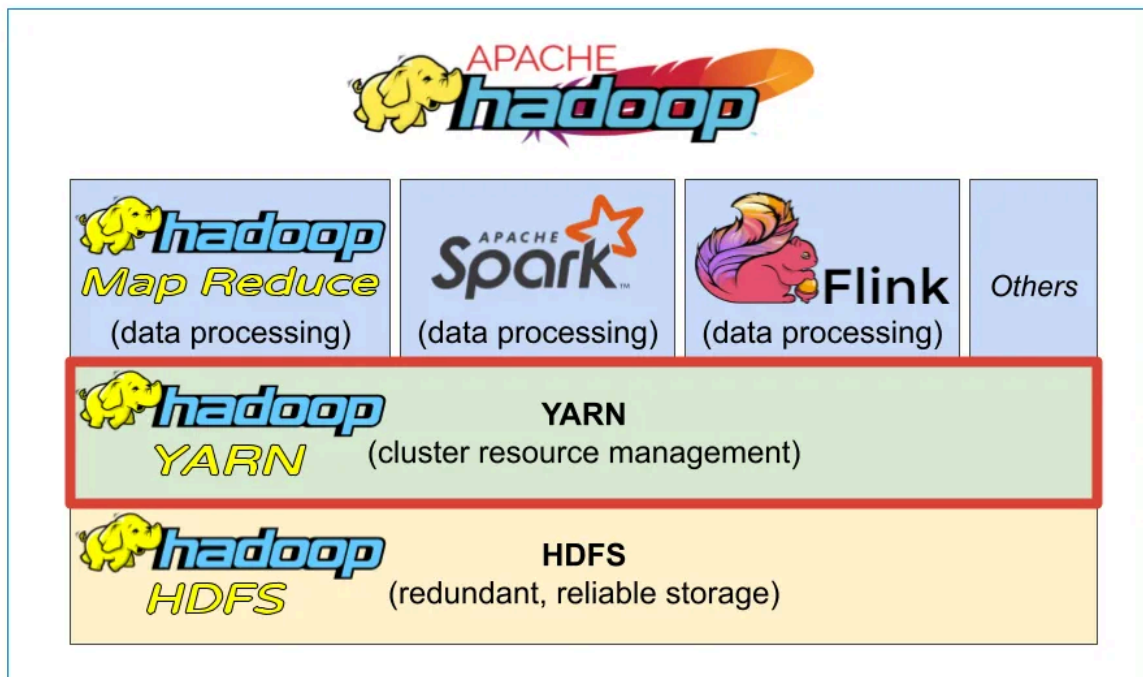
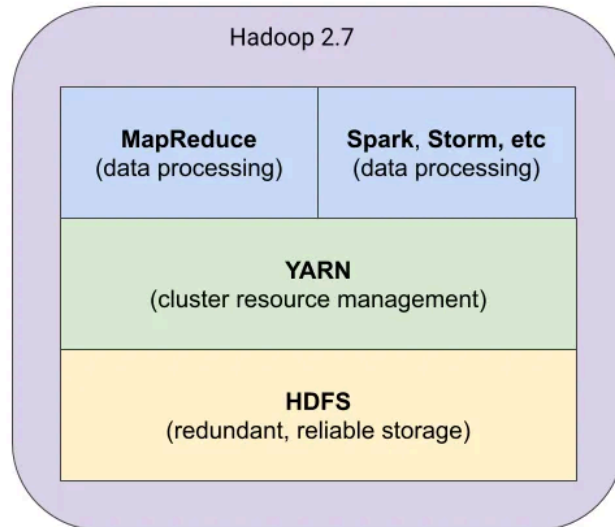
Before 2012

Users could write MapReduce programs using scripting languages



Since 2012

Users could work on multiple processing models in addition to MapReduce

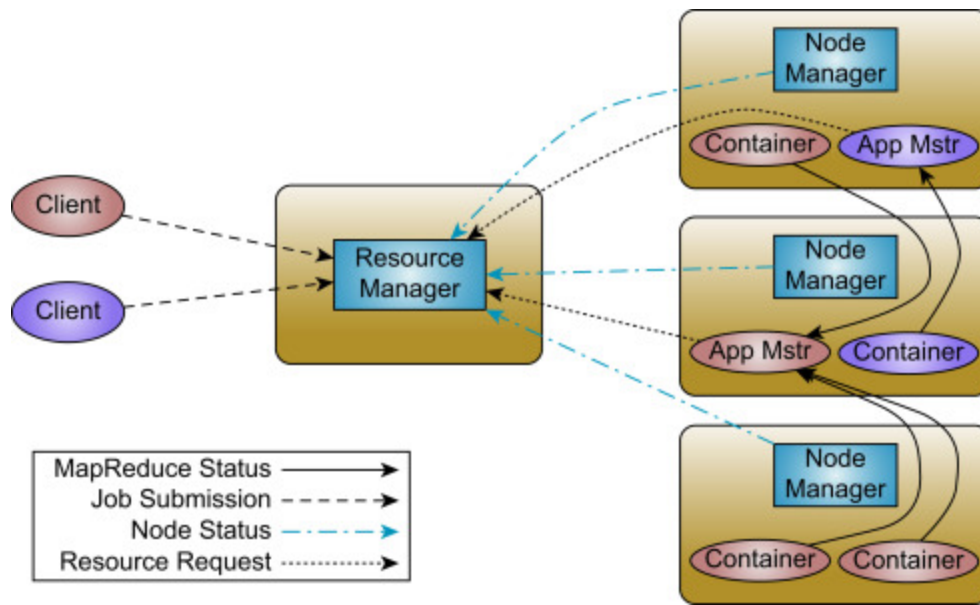


Benefit of YARN

- **Centralized Resource Management:** YARN provides a centralized resource management system that can dynamically allocate and manage resources for different frameworks/applications running on the Hadoop cluster.
- **Flexibility:** The scheduling and resource management capabilities are separated from the data processing component. This allows for running various types of data processing applications on a single cluster.
- **Better Cluster Utilization:** YARN's dynamic resource allocation ensures each application gets the resources it needs to run successfully without affecting other applications. Additionally, resources unutilized by one framework/application can be consumed by another.
- **Cost-Effective:** With YARN, one "do-it-all" Hadoop cluster can run a diverse set of workloads and support a variety of applications, making it a more efficient and cost-effective platform for big data processing.
- **Reduced Data Movement:** As there is no need to move data between Hadoop, YARN, and systems running on different clusters of computers, data motion is reduced.

Yarn Architecture

YARN can be thought of as the **operating system** for a Hadoop cluster. A cluster is a group of interconnected computers, working together to function as a unified system. This cluster represents a pool of shared resources, such as CPU, memory, disk space, and network bandwidth. YARN's role is to manage and distribute these resources among the various jobs running on the cluster. Just as an operating system manages a computer's resources and assigns them to competing processes, YARN ensures fair and efficient allocation of cluster resources to different jobs, optimizing performance and utilization.



It has two major responsibilities:

- Management of cluster resources such as compute, network, and memory
- Scheduling and monitoring of jobs

YARN's architecture comprises three primary components which helps it to fulfil its responsibilities

- **Resource Manager (RM)**
- **Node Manager (NM)**
- **Application Master (AM)**

Resource Manager

The **Resource Manager** is the master component of YARN, responsible for managing resources across the entire Hadoop cluster. It is deployed on a master node and acts as a central authority that allocates resources to various applications based on their needs.

The RM has two key sub-components:

1. Scheduler
2. Application Manager

Scheduler

The **YARN Scheduler** is a critical component of the **Resource Manager (RM)**, responsible for determining how resources (CPU, memory, etc.) are distributed across applications in the cluster. The Scheduler doesn't execute or monitor the application itself; its sole purpose is to allocate resources (called **containers**) to the various applications based on available resources and job priority.

The Scheduler needs to balance multiple factors:

- **Job requirements:** Different jobs have different resource requirements (e.g., one job might need a lot of memory, while another might need many CPU cores).
- **Resource availability:** The Scheduler allocates resources based on what's available across the cluster.
- **Fairness and prioritization:** The Scheduler ensures that no application monopolizes the resources, while also maintaining job priorities and fairness.

The Scheduler works on a **pluggable architecture**, meaning that Hadoop can use different types of scheduling algorithms depending on the cluster's requirements.

These include:

a) FIFO Scheduler (First In, First Out)

Overview: The **FIFO Scheduler** is the simplest scheduling algorithm. It processes jobs in the order in which they arrive. The job that is submitted first is given priority over all others, and only when it completes can the next job start. Each job gets all the required resources before the next one is scheduled.

How It Works:

1. Jobs are placed in a queue.
2. The first job in the queue is allocated resources until it completes.
3. Once the first job is done, the next job in the queue is scheduled and allocated resources.
4. This process continues until all jobs are processed.

b) Capacity Scheduler

Overview: The **Capacity Scheduler** allows multiple organizations, teams, or users to share a Hadoop cluster while ensuring that each group gets a specific percentage of resources. It is designed for **multi-tenancy** environments where different users need guaranteed access to certain amounts of cluster resources.

How It Works:

1. The cluster resources are divided into **queues** (logical partitions). Each queue is assigned a portion of the total resources, often represented as a percentage.
 - For example, 40% of the cluster might be allocated to a queue for the data science team, 30% for ETL jobs, and 30% for real-time analytics.
2. Each queue can run multiple jobs. If a queue isn't using all its resources, they can be temporarily borrowed by other queues.
3. Jobs within a queue are scheduled based on **FIFO** or **Fair Scheduling**.
4. If a queue exceeds its allocated resources, the jobs in that queue will be queued until enough resources become available.

c) Fair Scheduler

Overview: The **Fair Scheduler** ensures that all jobs running on the cluster receive a fair share of the available resources over time. It aims to **equalize resource allocation** among applications, so that long-running and short-running jobs are treated equitably, and no single job monopolizes the cluster's resources.

How It Works:

1. Jobs are placed in **pools**, and each pool is allocated an equal share of resources.
 - For example, if two pools are running, they each receive 50% of the resources, regardless of when their jobs were submitted.
2. Jobs in each pool are scheduled based on **priority** and **resource demands**.

3. The Scheduler dynamically adjusts resource allocations based on actual usage, giving underutilized jobs more resources until all pools receive their fair share.
4. **Preemption:** If a job has been waiting too long for resources while another job is using more than its fair share, the Scheduler can preempt the overusing job, reclaiming its resources and giving them to the waiting job.

d) Dominant Resource Fairness (DRF)

Overview: The **Dominant Resource Fairness (DRF)** algorithm extends the Fair Scheduler to consider multiple resource types (like CPU and memory) simultaneously. It ensures that no job monopolizes any one resource type, balancing multiple resource dimensions in an equitable way.

How It Works:

1. DRF calculates the dominant resource for each job—this is the resource (CPU or memory) that the job demands the most.
2. The algorithm then ensures that each job gets a fair share of its dominant resource. If one job is using too much of the dominant resource, it is slowed down so other jobs can get their fair share.
3. By balancing both CPU and memory, DRF prevents scenarios where a job may underutilize one resource (like CPU) while hogging another (like memory), which would otherwise lead to inefficient resource usage.

Application Manager

The Application Manager manages the **lifecycle of applications**. It handles job submission, launches the **Application Master** for each job, and monitors the job's overall status. The Application Manager works closely with the Application Master (explained below) to ensure job execution is running smoothly.

It also deals with failure recovery, ensuring that when an Application Master fails, it is restarted on another node.

Application Master (AM)

For each application (or job) submitted to YARN, an **Application Master** is created. The AM is responsible for managing the **lifecycle of the application**, including resource negotiation with the Resource Manager and task coordination across Node Managers.

The Application Master performs several critical tasks:

- **Requesting containers:** The AM negotiates with the Resource Manager to obtain containers for running tasks. It calculates the resource needs of its job and requests containers accordingly.
- **Task scheduling:** Once containers are allocated, the AM works with the Node Managers to schedule tasks to be executed within those containers.
- **Failure management:** The AM monitors the execution of tasks, detects task failures, and may request new containers to rerun failed tasks.
- **Progress reporting:** Throughout the job's execution, the AM reports the status of the job to the Resource Manager so that users can monitor job progress.

The **Application Master is job-specific**. Once a job is completed, the AM terminates, releasing all allocated resources back to the Resource Manager.

Node Manager

The **Node Manager** is the **per-node** agent that works on each node of the cluster. It is responsible for managing resources on a single node and reporting back to the Resource Manager.

Each Node Manager is responsible for:

- **Launching and monitoring containers:** Containers are the unit of execution in YARN, which encapsulate the CPU, memory, and other resources required for running tasks. The Node Manager is responsible for launching these containers and monitoring their resource usage (CPU, memory, disk I/O).
- **Reporting resource status:** The NM reports the status of resources on its node (such as available CPU and memory) back to the Resource Manager.

- **Health monitoring:** NMs monitor the health of their respective nodes and report issues like hardware failures to the Resource Manager.
- **Log management:** NMs handle the logging for each container running on their nodes and provide access to logs for troubleshooting or monitoring purposes.

The Node Manager is responsible for ensuring that tasks assigned to its node are completed successfully and that resources are efficiently utilized.

Yarn Containers

Containers are a fundamental concept in YARN's architecture. They represent the **logical units of resources** (CPU, memory, etc.) allocated by YARN for running an application's task.

When a job is submitted, the Resource Manager allocates containers based on the job's resource requirements, and these containers are launched on the appropriate Node Managers. A container provides an isolated environment for executing a specific task, ensuring that each task gets the resources it needs without interfering with others.

Key Points about YARN Containers:

- **Isolation:** Each container has its own assigned resources (CPU, memory) and operates independently of other containers. This prevents tasks from competing for resources.
- **Dynamic allocation:** Containers are dynamically allocated based on the needs of the application and the current availability of resources in the cluster.
- **Task execution:** Containers execute tasks as instructed by the Application Master. Each task runs within a container until completion, after which the container is released and can be reallocated to other tasks.

Yarn Workflow

Understanding how YARN orchestrates the execution of a job can be broken down into a series of steps. Let's walk through the workflow:

1. **Job Submission:**

- A client submits a job (e.g., a MapReduce job, Hive query, or Spark job) to the YARN Resource Manager. The job is treated as an **application** in YARN's terms.
- The Resource Manager receives the job and starts the **Application Manager**, which is responsible for managing the lifecycle of the job.

2. **Application Master Allocation:**

- The Resource Manager allocates a container for the **Application Master** to be launched. This Application Master is responsible for negotiating resources for the job and coordinating its execution.
- The Application Master then starts, initializes the job, and begins negotiating with the Resource Manager for additional containers to execute the job's tasks.

3. **Container Allocation:**

- Based on the job's requirements, the Application Master requests a set of containers from the Resource Manager.
- The Resource Manager evaluates the resource availability on various nodes in the cluster and allocates containers accordingly. These containers are launched on the appropriate nodes by the Node Managers.

4. **Task Execution:**

- Once containers are allocated, the Application Master communicates with the Node Managers on the nodes where the containers were assigned. The Node Managers then launch the tasks (e.g., Map or Reduce tasks for a MapReduce job) within the containers.
- The Node Manager monitors the tasks running in the containers and reports their status back to the Application Master.

5. **Monitoring and Failure Handling:**

- The Application Master continuously monitors the progress of tasks. If a task fails, it can request additional containers to restart the task.
- The Node Manager also reports back on the health of the node and container resource usage (e.g., CPU, memory, etc.).

6. **Job Completion:**

- Once all tasks are completed successfully, the Application Master notifies the Resource Manager of job completion and shuts down. The containers used by the job are released back to the Resource Manager for reallocation.
- The client is notified of the job's completion, and logs or output can be accessed for further analysis.